

Technical University
of Denmark

Department of Civil and Mechanical Engineering

41525 - Finite Element Methods

# Assignment 2

Group 8 (DOG):
Martin Hoel (s261610)
Ludvig August Grønlien-Magnussen (s261728)

Code, exercise 1 - Ludvig August Grønlien-Magnussen (s261728)
Code, exercise 3 - Ludvig August Grønlien-Magnussen (s261728)
Code, exercise 4 - Martin Hoel (s261610)

10.11.2026

# Table of Contents

**1 Exercise 1 - Equation solving** **2**
1.1 fea1.f90 . . . . . 2
1.2 plane42rect.f90 . . . . . 11

**2 Exercise 2 - Virtual Work Principle** **19**

**3 Exercise 3 - Isoparametric elements** **20**
3.1 fea3.f90 . . . . . 20
3.2 plane42.f90 . . . . . 31

**4 Exercise 4 - Eigenvalue analysis** **38**
4.1 fea.f90 . . . . . 38
4.2 plane42rectskewd.f90 . . . . . 50

# 1 Exercise 1 - Equation solving

## 1.1 fea1.f90

**Listing 1:** Finite element program

```
module fea

    !! This module contains procedures that are common to FEM-analysis

    implicit none
    save
    private
    public :: displ, initial, buildload, buildstiff, enforce, recover,
       compliance

contains

!
!
    ------------------------------------------------------------------------

!
    subroutine initial

        !! This subroutine is mainly used to allocate vectors and
           matrices

        use fedata
        use link1
        use plane42rect

! Hint for continuum elements:
        !integer, parameter :: mdim = 8
        integer, dimension(mdim) :: edof
        integer :: e, i, nen

        ! This subroutine computes the number of global equation,
        ! half bandwidth, etc and allocates global arrays.

        ! Calculate number of equations
        neqn = 2*nn

        ! Calculate half bandwidth (always, so it can be reported)
        bw = 0
        do e = 1, ne
            nen = element(e)%numnode
            do i = 1, nen
                edof(2*i-1) = 2 * element(e)%ix(i) - 1
                edof(2*i)   = 2 * element(e)%ix(i)
```

```
            end do
            bw = max(bw, maxval(edof(1:2*nen)) - minval(edof(1:2*nen))
               + 1)
        end do
        print '("ne␣=␣",␣i8,␣"␣␣␣neqn␣=␣",␣i8,␣"␣␣␣bw␣=␣",␣i6)', ne,
           neqn, bw

        if (.not. banded) then
            allocate (kmat(neqn, neqn))
        else
            allocate (kmat(bw, neqn))
        end if


        allocate (p(neqn), d(neqn))
        allocate (strain(ne, 3), stress(ne, 3))
        allocate (stress_eq(ne, 4))

        ! Initial stress and strain
        strain = 0
        stress = 0
        stress_eq = 0
    end subroutine initial
!
!
    ------------------------------------------------------------------------------


!
    subroutine displ

        !! This subroutine calculates displacements

        use fedata
        use numeth
        use processor

        !wp --> Precision

        integer :: e
        integer(8) :: c0, c1, c2, rate
        real(wp) :: t0, t1, t2
        real(wp), dimension(:), allocatable :: plotval

        ! Build load-vector
        call buildload

        ! Assembly (stiffness matrix + boundary conditions)
        call cpu_time(t0);  call system_clock(c0, rate)
        call buildstiff
```

```
86          call enforce
87          call cpu_time(t1);  call system_clock(c1)
88
89          ! Solution
90          if (.not. banded) then
91              call factor(kmat)
92              call solve(kmat, p)
93          else
94              call bfactor(kmat)
95              call bsolve(kmat, p)
96          end if
97          call cpu_time(t2);  call system_clock(c2)
98
99          print '("CPU time      - Assembly: ", f10.4, " s    Solve: ", f10
                .4, " s")', &
100             t1 - t0, t2 - t1
101         print '("System clock - Assembly: ", f10.4, " s    Solve: ", f10
                .4, " s")', &
102             real(c1 - c0, wp) / real(rate, wp), real(c2 - c1, wp) /
                    real(rate, wp)
103
104         ! Open file on unit 21 (unit number is an arbitrary file handle
                );
105         ! Save results for the report (one line per run)
106         open (21, file='timing.dat', position='append')
107         write (21, '(a20, 1x, l1, 3(1x, i8), 2(1x, es12.4))') &
108             trim(filename), banded, ne, neqn, bw, t1-t0, t2-t1
109         close (21)
110
111
112         ! Transfer results
113         d(1:neqn) = p(1:neqn)
114
115         ! Recover stress
116         call recover
117
118         ! Output results
119         call output
120
121         ! Flush the screen buffer so output appears immediately (and is
                not lost if the program crashes)
122         flush(6)
123
124         ! Plot deformed structure
125         call plotmatlabdef('Deformed')
126
127         ! Plot element values
128         allocate (plotval(ne))
129         do e = 1, ne
```

```
            if (element(e)%id == 1) then
                plotval(e) = stress(e,1)
            else if (element(e)%id == 2) then
                plotval(e) = stress(e,3)
            end if
        end do
        call plotmatlabeval('Shear stress',plotval)
        call plotmatlabeval('von Mises', stress_eq(:, 1))
        call plotmatlabevec('Principal stresses', stress_eq(:, 2),
           stress_eq(:, 3), stress_eq(:, 4))

    end subroutine displ

!
!
    ------------------------------------------------------------------------

!
    subroutine compliance

        !! This subroutine calculates compliance

        use fedata

        real(wp) :: c

        c = dot_product(p, d)
        write (*, '("Compliance C = p^T d = ",es15.8)') c
        open (20, file='compliance.dat', position='append')
        write (20, '(i8, 1x, i8, 1x, es20.10)') ne, neqn, c
        close (20)

    end subroutine compliance


!
!
    ------------------------------------------------------------------------

!
    subroutine buildload

        !! This subroutine builds the global load vector

        use fedata
        use plane42rect

! Hint for continuum elements:

```

```
174         integer :: i, j, e, nen, idof, eface
175         real(wp) :: fe, thk
176         integer, dimension(mdim) :: edof
177         real(wp), dimension(mdim) :: xe
178         real(wp), dimension(mdim) :: re
179 
180         ! Build load vector
181         p(1:neqn) = 0
182         do i = 1, np
183             select case(int(loads(i, 1)))
184             case( 1 )
185                 ! Build nodal load contribution
186                 idof = int(2*(loads(i,2)-1) + loads(i,3))
187                 p(idof) = p(idof) + loads(i, 4)
188             case( 2 )
189                 ! Build uniformly distributed surface (pressure) load
                        contribution
190                 e = int(loads(i, 2))
191                 eface = int(loads(i,3))
192                 fe    = loads(i,4)
193                 thk   = mprop(element(e)%mat)%thk
194                 nen   = element(e)%numnode
195 
196                 do j = 1, nen
197                     xe(2*j-1) = x(element(e)%ix(j),1)
198                     xe(2*j  ) = x(element(e)%ix(j),2)
199                     edof(2*j-1) = 2 * element(e)%ix(j) - 1
200                     edof(2*j)   = 2 * element(e)%ix(j)
201                 end do
202                 call plane42rect_re(xe, eface, fe, thk, re)
203                 p(edof(1: 2*nen)) = p(edof(1: 2*nen)) + re(1:2*nen)
204             case default
205                 print *, 'ERROR␣in␣fea/buildload␣2'
206                 print *, 'Load␣type␣not␣known'
207                 stop
208             end select
209         end do
210     end subroutine buildload
211 !
212 !
    ----------------------------------------------------------------------------

213 !
214     subroutine buildstiff
215 
216         !! This subroutine builds the global stiffness matrix from
217         !! the local element stiffness matrices
218 
219         use fedata
```

```
220         use link1
221         use plane42rect
222 
223         integer :: e, i, j
224         integer :: nen
225 ! Hint for system matrix in band form:
226         integer :: irow, icol
227         integer, dimension(mdim) :: edof
228         real(wp), dimension(mdim) :: xe
229         real(wp), dimension(mdim, mdim) :: ke
230 ! Hint for modal analysis:
231             real(wp), dimension(mdim, mdim) :: me
232         real(wp) :: young, area
233 ! Hint for modal analysis and continuum elements:
234         real(wp) :: nu, dens, thk
235 
236         ! Reset stiffness matrix
237         kmat = 0
238         !if (.not. banded) then
239         !    kmat = 0
240         !else
241         !    kmat = 0
242         !end if
243 
244         do e = 1, ne
245 
246             ! Find coordinates and degrees of freedom
247             nen = element(e)%numnode
248             do i = 1, nen
249                 xe(2*i-1) = x(element(e)%ix(i),1)
250                 xe(2*i  ) = x(element(e)%ix(i),2)
251                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
252                 edof(2*i)   = 2 * element(e)%ix(i)
253             end do
254 
255             ! Gather material properties and find element stiffness
                    matrix
256             select case( element(e)%id )
257             case( 1 )
258                 young = mprop(element(e)%mat)%young
259                 area  = mprop(element(e)%mat)%area
260                 call link1_ke(xe, young, area, ke)
261             case( 2 )
262                 young = mprop(element(e)%mat)%young
263                 nu = mprop(element(e)%mat)%nu
264                 thk = mprop(element(e)%mat)%thk
265                 call plane42rect_ke(xe, young, nu, thk, ke)
266             end select
267 
```

```
268             ! Assemble into global matrix
269             if (.not. banded) then
270                 do i = 1, 2*nen
271                     do j = 1, 2*nen
272                         kmat(edof(i), edof(j)) = kmat(edof(i), edof(j))
                                + ke(i, j)
273                     end do
274                 end do
275 ! Hint: Can you eliminate the loops above by using a different Fortran
    array syntax?
276             else
277                 do i = 1, 2*nen
278                     do j = 1, 2*nen
279                     if (edof(j) >= edof(i)) then
280                         kmat(edof(j)-edof(i)+1, edof(i)) = kmat(edof(j)
                            -edof(i)+1, edof(i)) + ke(i, j)
281                     end if
282                     end do
283                 end do
284             end if
285         end do
286     end subroutine buildstiff
287 !
288 !
    ------------------------------------------------------------------------------

289 !
290     subroutine enforce
291
292         !! This subroutine enforces the support boundary conditions
293
294         use fedata
295
296         integer :: i, idof, k, r
297         real(wp) :: penal
298
299         ! Correct for supports
300         if (.not. banded) then
301                 if (.not. penalty) then
302                     do i = 1, nb
303                         idof = int(2*(bound(i,1)-1) + bound(i,2))
304                         p(1:neqn) = p(1:neqn) - kmat(1:neqn, idof) *
                            bound(i, 3)
305                         p(idof) = bound(i, 3)
306                         kmat(1:neqn, idof) = 0
307                         kmat(idof, 1:neqn) = 0
308                         kmat(idof, idof) = 1
309                     end do
310             else
```

```
311                 penal = penalty_fac*maxval(kmat)
312                 do i = 1, nb
313                     idof = int(2*(bound(i,1)-1) + bound(i,2))
314                     kmat(idof, idof) = kmat(idof, idof) + penal
315                     p(idof) = penal * bound(i, 3)
316                 end do
317             end if
318         else
319             if (.not. penalty) then
320                 do i = 1, nb
321                     idof = int(2*(bound(i,1)-1) + bound(i,2))
322
323                     do k = (idof-min(idof-1,bw-1)), (idof-1)
324                         P(k) = P(k) - kmat(idof+1-k, k) * bound(i, 3)
325                         kmat(idof+1-k, k) = 0
326                     end do
327
328                     do r = 2, min(bw, neqn - idof + 1)
329                         P(idof+r-1) = P(idof+r-1) - kmat(r, idof) *
                               bound(i, 3)
330                         kmat(r, idof) = 0
331                     end do
332
333                     p(idof) = bound(i, 3)
334                     kmat(1, idof) = 1
335                 end do
336             else
337                 penal = penalty_fac*maxval(kmat)
338                 do i = 1, nb
339                     idof = int(2*(bound(i,1)-1) + bound(i,2))
340                     kmat(1, idof) = kmat(1, idof) + penal
341                     p(idof) = penal * bound(i, 3)
342                 end do
343             end if
344         end if
345     end subroutine enforce
346 !
347 !
    ----------------------------------------------------------------------------

348 !
349     subroutine recover
350
351         !! This subroutine recovers the element stress, element strain,
352         !! and nodal reaction forces
353
354         use fedata
355         use link1
356         use plane42rect
```

```
357
358         integer :: e, i, nen
359         integer :: edof(mdim)
360         real(wp), dimension(mdim) :: xe, de
361         real(wp), dimension(mdim, mdim) :: ke
362         real(wp) :: young, area
363 ! Hint for continuum elements:
364         real(wp):: nu, dens, thk, sigma_vm, sigma_1, sigma_2, psi
365         real(wp), dimension(3) :: estrain, estress
366
367         ! Reset force vector
368         p = 0
369
370         do e = 1, ne
371
372             ! Find coordinates etc...
373             nen = element(e)%numnode
374             do i = 1,nen
375                 xe(2*i-1) = x(element(e)%ix(i), 1)
376                 xe(2*i)   = x(element(e)%ix(i), 2)
377                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
378                 edof(2*i)   = 2 * element(e)%ix(i)
379                 de(2*i-1) = d(edof(2*i-1))
380                 de(2*i)   = d(edof(2*i))
381             end do
382
383             ! Find stress and strain
384             select case( element(e)%id )
385             case( 1 )
386                 young = mprop(element(e)%mat)%young
387                 area  = mprop(element(e)%mat)%area
388                 call link1_ke(xe, young, area, ke)
389                 p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke(1:2*nen
                        ,1:2*nen), de(1:2*nen))
390                 call link1_ss(xe, de, young, estress, estrain)
391                 stress(e, 1:3) = estress
392                 strain(e, 1:3) = estrain
393                 case( 2 )
394                     young = mprop(element(e)%mat)%young
395                     nu = mprop(element(e)%mat)%nu
396                     thk = mprop(element(e)%mat)%thk
397                     call plane42rect_ke(xe, young, nu, thk, ke)
398                     p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke
                            (1:2*nen,1:2*nen), de(1:2*nen))
399                     call plane42rect_ss(xe, de, young, nu, estress,
                            estrain)
400                     stress(e, 1:3) = estress
401                     strain(e, 1:3) = estrain
402
```

```
403                     sigma_vm = sqrt((estress(1)**2) + (estress(2)**2) -
                            (estress(1)*estress(2)) + 3*(estress(3)**2))
404                     sigma_1 = 0.5 * (estress(1) + estress(2))+sqrt(((
                           estress(1)-estress(2))/2)**2 + (estress(3)**2))
405                     sigma_2 = 0.5 * (estress(1) + estress(2))-sqrt(((
                           estress(1)-estress(2))/2)**2 + (estress(3)**2))
406                     psi = atan2(((-2*estress(3))), ((estress(1)-estress
                           (2))))/2.0_wp
407
408                     stress_eq(e, 1) = sigma_vm
409                     stress_eq(e, 2) = sigma_1
410                     stress_eq(e, 3) = sigma_2
411                     stress_eq(e, 4) = psi
412             end select
413         end do
414     end subroutine recover
415
416
417 end module fea
```

## 1.2 plane42rect.f90

**Listing 2:** Finite element program

```
1  module plane42rect
2
3      !! This module contains subroutines specific to the plane42 element
4      !!
5      !! The plane42 element has 4 nodes. Each node has 2 degrees-of-
           freedom,
6      !! namely, displacement along the \(x\)- and \(y\)-coordinate
           directions.
7      !!
8      !! The nodes are numbered counter-clockwise as indicated below. The
9      !! figure also shows how the element edges are labelled. For
           example,
10     !! edge 1 is the edge between element node 1 and 2.
11     !!
12     !!        N4     E3     N3
13     !!           o------o
14     !!           |      |
15     !!        E4 |      | E2
16     !!           |      |
17     !!           o------o
18     !!        N1     E1     N2
19     !!
20     !!
21     !! `N1` = element node 1, `N2` = element node 2, etc
22     !! `E1` = element face 1, `E2` = element face 2, etc
```

```
23
24      use types
25      implicit none
26      save
27
28      private
29      public :: plane42rect_ke, plane42rect_re, plane42rect_ss
30
31  contains
32
33      subroutine plane42rect_ke(xe, young, nu, thk, ke)
34
35          !! This subroutine constructs the stiffness matrix for
36          !! a rectangular 4-noded quad element.
37
38          real(wp), intent(in) :: young
39              !! Young's Modulus for this element
40          real(wp), intent(in) :: nu
41              !! Poisson's Ratio for this element
42          real(wp), intent(in) :: thk
43              !! Thickness of this element
44          real(wp), dimension(:), intent(in) :: xe
45              !! Nodal coordinates of this element in undeformed
                   configuration
46              !!
47              !! * `xe(1:2)` = \((x,y)\)-coordinates of element node 1
48              !! * `xe(3:4)` = \((x,y)\)-coordinates of element node 1
49              !! * `xe(5:6)` = \((x,y)\)-coordinates of element node 2
50              !! * `xe(7:8)` = \((x,y)\)-coordinates of element node 2
51              !!
52              !! See also [[plane42rect]]
53          real(wp), dimension(:,:), intent(out) :: ke
54              !! Stiffness matrix
55
56          real(wp) :: cmat(3,3), fact, aa, bb
57          real(wp) :: t1, t2, t3, t4, t5, t6, t9, t11, t13, t15, &
58                      t16, t17, t18, t19, t22, t25, t29, t33, &
59                      t34, t38, t42, t43, t47, t51, t55, t56, &
60                      t57, t58, t59, t62, t66, t70, t74, t78, &
61                      t82, t86, t90, t94, t98, t102, t106, t110
62          real(wp) :: d11, d12, d13, d22, d23, d33
63
64          aa = (xe(3)-xe(1))/2
65          bb = (xe(8)-xe(2))/2
66
67          ! build constitutive matrix (plane stress)
68          cmat = 0
69          fact = young/(1-nu**2)
70          cmat(1,1) = fact
```

```
71          cmat(1,2) = fact*nu
72          cmat(2,1) = fact*nu
73          cmat(2,2) = fact
74          cmat(3,3) = fact*(1-nu)/2
75
76          d11 = cmat(1,1)
77          d12 = cmat(1,2)
78          d13 = cmat(1,3)
79          d22 = cmat(2,2)
80          d23 = cmat(2,3)
81          d33 = cmat(3,3)
82
83          t1 = bb**2
84          t2 = t1*d11
85          t3 = 2*t2
86          t4 = aa**2
87          t5 = t4*d33
88          t6 = 2*t5
89          t9 = 3*d13*aa*bb
90          t11 = 1/aa
91          t13 = 1/bb
92          t15 = (t3+t6+t9)*t11*t13/6
93          t16 = d13*t1
94          t17 = 4*t16
95          t18 = t4*d23
96          t19 = 4*t18
97          t22 = 3*d12*aa*bb
98          t25 = 3*aa*d33*bb
99          t29 = (t17+t19+t22+t25)*t11*t13/12
100         t33 = (-t3+t5)*t11*t13/6
101         t34 = 2*t18
102         t38 = (-t17+t34+t22-t25)*t11*t13/12
103         t42 = (t2+t5+t9)*t11*t13/6
104         t43 = 2*t16
105         t47 = (t43+t34+t22+t25)*t11*t13/12
106         t51 = (-t2+t6)*t11*t13/6
107         t55 = (-t43+t19+t22-t25)*t11*t13/12
108         t56 = d33*t1
109         t57 = 2*t56
110         t58 = t4*d22
111         t59 = 2*t58
112         t62 = 3*aa*d23*bb
113         t66 = (t57+t59+t62)*t11*t13/6
114         t70 = (-t17+t34-t22+t25)*t11*t13/12
115         t74 = (-t57+t58)*t11*t13/6
116         t78 = (t56+t58+t62)*t11*t13/6
117         t82 = (-t43+t19-t22+t25)*t11*t13/12
118         t86 = (-t56+t59)*t11*t13/6
119         t90 = (t3+t6-t9)*t11*t13/6
```

```
      t94 = (t17+t19-t22-t25)*t11*t13/12
      t98 = (t2+t5-t9)*t11*t13/6
      t102 = (t43+t34-t22-t25)*t11*t13/12
      t106 = (t57+t59-t62)*t11*t13/6
      t110 = (t56+t58-t62)*t11*t13/6
      ke(1,1) = t15
      ke(1,2) = t29
      ke(1,3) = t33
      ke(1,4) = t38
      ke(1,5) = -t42
      ke(1,6) = -t47
      ke(1,7) = -t51
      ke(1,8) = -t55
      ke(2,1) = t29
      ke(2,2) = t66
      ke(2,3) = t70
      ke(2,4) = t74
      ke(2,5) = -t47
      ke(2,6) = -t78
      ke(2,7) = -t82
      ke(2,8) = -t86
      ke(3,1) = t33
      ke(3,2) = t70
      ke(3,3) = t90
      ke(3,4) = t94
      ke(3,5) = -t51
      ke(3,6) = -t82
      ke(3,7) = -t98
      ke(3,8) = -t102
      ke(4,1) = t38
      ke(4,2) = t74
      ke(4,3) = t94
      ke(4,4) = t106
      ke(4,5) = -t55
      ke(4,6) = -t86
      ke(4,7) = -t102
      ke(4,8) = -t110
      ke(5,1) = -t42
      ke(5,2) = -t47
      ke(5,3) = -t51
      ke(5,4) = -t55
      ke(5,5) = t15
      ke(5,6) = t29
      ke(5,7) = t33
      ke(5,8) = t38
      ke(6,1) = -t47
      ke(6,2) = -t78
      ke(6,3) = -t82
      ke(6,4) = -t86
```

```
        ke(6,5) = t29
        ke(6,6) = t66
        ke(6,7) = t70
        ke(6,8) = t74
        ke(7,1) = -t51
        ke(7,2) = -t82
        ke(7,3) = -t98
        ke(7,4) = -t102
        ke(7,5) = t33
        ke(7,6) = t70
        ke(7,7) = t90
        ke(7,8) = t94
        ke(8,1) = -t55
        ke(8,2) = -t86
        ke(8,3) = -t102
        ke(8,4) = -t110
        ke(8,5) = t38
        ke(8,6) = t74
        ke(8,7) = t94
        ke(8,8) = t106

        ke = ke*thk
    end subroutine plane42rect_ke
!
!
    ----------------------------------------------------------------------------

!
    subroutine plane42rect_re(xe, eface, fe, thk, re)

        !! This subroutine computes the element load vector due
        !! to surface traction (traction is always perpendicular
        !! to element face).

        integer, intent(in) :: eface
            !! Element face where traction (pressure) is applied

        real(wp), intent(in) :: fe
            !! Value of surface traction (pressure)
        real(wp), intent(in) :: thk
            !! Thickness of this element
        real(wp), dimension(:), intent(in) :: xe
            !! Nodal coordinates of this element in undeformed
               configuration (see also [[plane42rect_ke]])
        real(wp), intent(out) :: re(8)
            !! Element force vector
            !!
            !! * `re(1:2)` = \((f_x^1, f_y^1)\) force at element node 1
                in \(x\)- and y-direction
```

```
214             !! * `re(3:4)` = \((f_x^2, f_y^2)\) force at element node 1
                    in \(x\)- and y-direction
215             !! * etc...
216         real(wp) :: aa, bb, nface(2,8), f(2)
217
218         aa = (xe(3)-xe(1))/2
219         bb = (xe(8)-xe(2))/2
220
221         nface = 0
222         f = 0
223         if (eface == 1) then
224             f = [0.0_wp, fe]
225             nface(1,1) = aa
226             nface(1,3) = aa
227             nface(2,2) = aa
228             nface(2,4) = aa
229         elseif (eface == 2) then
230             f = [-fe, 0.0_wp]
231             nface(1,3) = bb
232             nface(1,5) = bb
233             nface(2,4) = bb
234             nface(2,6) = bb
235         elseif (eface == 3) then
236             f = [0.0_wp, -fe]
237             nface(1,5) = aa
238             nface(1,7) = aa
239             nface(2,6) = aa
240             nface(2,8) = aa
241         elseif (eface == 4) then
242             f = [fe, 0.0_wp]
243             nface(1,1) = bb
244             nface(1,7) = bb
245             nface(2,2) = bb
246             nface(2,8) = bb
247         endif
248         re = matmul(transpose(nface), f) * thk
249     end subroutine plane42rect_re
250 !
251 !
    ------------------------------------------------------------------------

252 !
253     subroutine plane42rect_ss(xe, de, young, nu, estress, estrain)
254
255         !! This subrotuine computes the element stress and strain (The
               location inside the element
256         !! where stress and and strain is evaluated, is defined inside
               the subroutine).
257
```

```
258         real(wp), intent(in) :: young
259             !! Young's Modulus for this element
260         real(wp), intent(in) :: nu
261             !! Poisson's Ratio for this element
262         real(wp), dimension(:), intent(in)  :: xe
263             !! Nodal coordinates of this element in undeformed
                configuration (see also [[plane42rect_ke]])
264         real(wp), dimension(:), intent(in)  :: de
265             !! Displacement field
266             !!
267             !! * `de(1:2)` = displacement of element node 1 along \(x\)
                - and \(y\)-axis, respectively
268             !! * `de(3:4)` = displacement of element node 2 along \(x\)
                - and \(y\)-axis, respectively
269             !! * etc...
270         real(wp), dimension(:), intent(out) :: estress
271             !! Stress at a point inside the element
272             !!
273             !! * `estress(1)` = \(\sigma_{11}\)
274             !! * `estress(2)` = \(\sigma_{22}\)
275             !! * `estress(3)` = \(\sigma_{12}\)
276         real(wp), dimension(:), intent(out) :: estrain
277             !! Strain at a point inside the element
278             !!
279             !! * `estrain(1)` = \(\epsilon_{11}\)
280             !! * `estrain(2)` = \(\epsilon_{22}\)
281             !! * `estrain(3)` = \(\epsilon_{12}\)
282
283         ! New variables
284         real(wp) :: bmat(3, 8), cmat(3, 3)
285         real(wp) :: aa, bb, dndx(4), dndy(4), fact
286         integer :: i
287
288         ! Build strain-displacement matrix
289
290         aa = (xe(3)-xe(1))/2
291         bb = (xe(8)-xe(2))/2
292         dndx = [-1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp]/(4*aa)
293         dndy = [-1.0_wp, -1.0_wp, 1.0_wp, 1.0_wp]/(4*bb)
294
295         bmat = 0
296         do i = 1, 4
297             bmat(1,2*i-1) = dndx(i)
298             bmat(2,2*i) = dndy(i)
299             bmat(3,2*i-1) = dndy(i)
300             bmat(3,2*i) = dndx(i)
301         end do
302
303         ! Compute element strain
```

```
304         estrain = matmul(bmat, de)
305
306         ! Build constitutive matrix (plane stress)
307         cmat = 0
308         fact = young/(1.0_wp-nu**2)
309         cmat(1,1) = fact
310         cmat(1,2) = fact*nu
311         cmat(2,1) = fact*nu
312         cmat(2,2) = fact
313         cmat(3,3) = fact*(1.0_wp-nu)/2
314
315         ! Compute element stress
316         estress = matmul(cmat, estrain)
317
318         ! Compute principal stress and direction
319         ! Does this in fea.f90 to not change the subroutine
320
321     end subroutine plane42rect_ss
322
323 end module plane42rect
```

# 2 Exercise 2 - Virtual Work Principle

No code used

# 3 Exercise 3 - Isoparametric elements

## 3.1 fea3.f90

**Listing 3:** Finite element program

```
module fea

    !! This module contains procedures that are common to FEM-analysis

    implicit none
    save
    private
    public :: displ, initial, buildload, buildstiff, enforce, recover,
       compliance, tip_deflection

contains

!
!
    ------------------------------------------------------------------------

!
    subroutine initial

        !! This subroutine is mainly used to allocate vectors and
           matrices

        use fedata
        use link1
        use plane42

! Hint for continuum elements:
        !integer, parameter :: mdim = 8
        integer, dimension(mdim) :: edof
        integer :: e, i, nen

        ! This subroutine computes the number of global equation,
        ! half bandwidth, etc and allocates global arrays.

        ! Calculate number of equations
        neqn = 2*nn

        ! Calculate half bandwidth (always, so it can be reported)
        bw = 0
        do e = 1, ne
            nen = element(e)%numnode
            do i = 1, nen
                edof(2*i-1) = 2 * element(e)%ix(i) - 1
                edof(2*i)   = 2 * element(e)%ix(i)
```

```
            end do
            bw = max(bw, maxval(edof(1:2*nen)) - minval(edof(1:2*nen))
               + 1)
        end do
        print '("ne␣=␣",␣i8,␣"␣␣␣neqn␣=␣",␣i8,␣"␣␣␣bw␣=␣",␣i6)', ne,
           neqn, bw

        if (.not. banded) then
            allocate (kmat(neqn, neqn))
        else
            allocate (kmat(bw, neqn))
        end if


        allocate (p(neqn), d(neqn))
        allocate (strain(ne, 3), stress(ne, 3))
        allocate (stress_eq(ne, 4))

        ! Initial stress and strain
        strain = 0
        stress = 0
        stress_eq = 0
    end subroutine initial
!
!
    ------------------------------------------------------------------------------


!
    subroutine displ

!This subroutine calculates displacements

        use fedata
        use numeth
        use processor

        !wp --> Precision

        integer :: e
        integer(8) :: c0, c1, c2, rate
        real(wp) :: t0, t1, t2
        real(wp), dimension(:), allocatable :: plotval
        real(wp) :: A_coord(2)      ! coordinates of point A
        integer  :: nodeA           ! node closest to point A
        real(wp) :: vmA             ! von Mises stress at point A
        integer  :: n_el            ! number of elements that share
           node A
        real(wp) :: delta
        integer  :: n_end
```

```
85
86          ! Build load-vector
87          call buildload
88
89          ! Assembly (stiffness matrix + boundary conditions)
90          call cpu_time(t0);  call system_clock(c0, rate)
91          call buildstiff
92          call enforce
93          call cpu_time(t1);  call system_clock(c1)
94
95          ! Solution
96          if (.not. banded) then
97              call factor(kmat)
98              call solve(kmat, p)
99          else
100             call bfactor(kmat)
101             call bsolve(kmat, p)
102         end if
103         call cpu_time(t2);  call system_clock(c2)
104
105         print '("CPU time      - Assembly: ", f10.4, " s    Solve: ", f10
                .4, " s")', &
106             t1 - t0, t2 - t1
107         print '("System clock - Assembly: ", f10.4, " s    Solve: ", f10
                .4, " s")', &
108             real(c1 - c0, wp) / real(rate, wp), real(c2 - c1, wp) /
                    real(rate, wp)
109
110         ! Open file on unit 21 (unit number is an arbitrary file handle
                );
111         ! Save results for the report (one line per run)
112         open (21, file='timing.dat', position='append')
113         write (21, '(a20, 1x, l1, 3(1x, i8), 2(1x, es12.4))') &
114             trim(filename), banded, ne, neqn, bw, t1-t0, t2-t1
115         close (21)
116
117
118         ! Transfer results
119         d(1:neqn) = p(1:neqn)
120
121         ! Recover stress
122         call recover
123
124         A_coord = [64.45_wp, 15.20_wp]
125         ! Node closets to A. minloc finds the index with the lowest
                value
126         nodeA = minloc((x(1:nn,1)-A_coord(1))**2 + (x(1:nn,2)-A_coord
                (2))**2, dim=1)
127
```

```
128         !print*, 'Node at A', nodeA,x(nodeA,1), x(nodeA,2) !Printed out
                in the terminal that A is node 562
129
130         ! Average von Mises of the elements that contain node A
131         vmA  = 0.0_wp
132         n_el = 0
133         ! Loop that finds the elements close to Node A
134         do e = 1, ne
135             ! element(e)%ix is the list of the nodes
136             if (any(element(e)%ix == nodeA)) then   ! Compares the
                    values in the list to the nodeA (562). Gives a list of
                    True or False
137                 vmA  = vmA + stress_eq(e,1)      ! Von mises for the
                        element e
138                 n_el = n_el + 1                  ! Counts how many
                        elements that contains node A
139             end if
140         end do
141         vmA = vmA / n_el                         ! Turns the sum value
                to the avereage, since each element gives its own stress at
                A.
142
143         print *, 'von Mises in point A:', vmA, 'MPa (average of', n_el,
                'elements)'
144         !i0  (i = integer, 0 means the width), f6.2 = 6 digits of the
                number with 2 decimals
145         print '(/"Point A is node ", i0, " at (x,y) = (", f6.2, ", ",
                f6.2, ") mm")', nodeA, x(nodeA,1), x(nodeA,2)
146         print '("Node ", i0, " is shared by ", i0, " elements")', nodeA
                , n_el
147         print '("von Mises stress at A (average of these elements) = ",
                f7.2, " MPa"/)', vmA
148         print '("Maximum von Mises stress = ", f7.2, " MPa")', maxval(
                stress_eq(:,1))
149
150
151
152
153         ! Output results
154         call output
155
156         ! Flush the screen buffer so output appears immediately (and is
                not lost if the program crashes)
157         flush(6)
158
159         ! Plot deformed structure
160         call plotmatlabdef('Deformed')
161
162         ! Plot element values
```

```
163         allocate (plotval(ne))
164         do e = 1, ne
165             if (element(e)%id == 1) then
166                 plotval(e) = stress(e,1)
167             else if (element(e)%id == 2) then
168                 plotval(e) = stress(e,3)
169             end if
170         end do
171         call plotmatlabeval('Shear stress',plotval)
172         call plotmatlabeval('von Mises', stress_eq(:, 1))
173         call plotmatlabevec('Principal stresses', stress_eq(:, 2),
               stress_eq(:, 3), stress_eq(:, 4))
174         call tip_deflection(300.0_wp, delta, n_end)
175     print '("Tip deflection delta = ", f8.4, " mm  (", i0, " nodes)")',
            delta, n_end
176
177     end subroutine displ
178
179 !
    ----------------------------------------------------------------------------
180
181     subroutine tip_deflection(x_tip, delta, n_end)
182
183         use fedata
184
185         real(wp), intent(in)  :: x_tip
186         real(wp), intent(out) :: delta
187         integer,  intent(out) :: n_end
188         integer  :: i
189         real(wp) :: sum_uy
190
191         sum_uy = 0.0_wp
192         n_end  = 0
193         ! Loop goes through all the nodes and picks out the nodes at
               the end, where x = 300
194         do i = 1, nn
195         ! Uses a small tolerance so x(i,1) almost equals to 300 because
               == would skip the nodes that aren't exactly 300
196             if (abs(x(i,1) - x_tip) < 1.0e-6_wp) then
197                 sum_uy = sum_uy + d(2*i)
198                 n_end  = n_end + 1
199             end if
200         end do
201         delta = -sum_uy / n_end
202
203     end subroutine tip_deflection
204
205 !
```

```
206 !
    ------------------------------------------------------------------------------

207 !
208     subroutine compliance
209
210         !! This subroutine calculates compliance
211
212         use fedata
213
214         real(wp) :: c
215
216         c = dot_product(p, d)
217         write (*, '("Compliance␣C␣=␣p^T␣d␣=␣",␣es15.8)') c
218         open (20, file='compliance.dat', position='append')
219         write (20, '(i8,␣1x,␣i8,␣1x,␣es20.10)') ne, neqn, c
220         close (20)
221
222     end subroutine compliance
223
224
225 !
226 !
    ------------------------------------------------------------------------------

227 !
228     subroutine buildload
229
230         !! This subroutine builds the global load vector
231
232         use fedata
233         use plane42
234
235 ! Hint for continuum elements:
236
237         integer :: i, j, e, nen, idof, eface
238         real(wp) :: fe, thk
239         integer, dimension(mdim) :: edof
240         real(wp), dimension(mdim) :: xe
241         real(wp), dimension(mdim) :: re
242
243         ! Build load vector
244         p(1:neqn) = 0
245         do i = 1, np
246             select case(int(loads(i, 1)))
247             case( 1 )
248                 ! Build nodal load contribution
249                 idof = int(2*(loads(i,2)-1) + loads(i,3))
250                 p(idof) = p(idof) + loads(i, 4)
```

```
            case( 2 )
                ! Build uniformly distributed surface (pressure) load
                   contribution
                e = int(loads(i, 2))
                eface = int(loads(i,3))
                fe    = loads(i,4)
                thk   = mprop(element(e)%mat)%thk
                nen   = element(e)%numnode

                do j = 1, nen
                    xe(2*j-1) = x(element(e)%ix(j),1)
                    xe(2*j  ) = x(element(e)%ix(j),2)
                    edof(2*j-1) = 2 * element(e)%ix(j) - 1
                    edof(2*j)   = 2 * element(e)%ix(j)
                end do
                call plane42_re(xe, eface, fe, thk, re)
                p(edof(1: 2*nen)) = p(edof(1: 2*nen)) + re(1:2*nen)
            case default
                print *, 'ERROR in fea/buildload 2'
                print *, 'Load type not known'
                stop
            end select
        end do
    end subroutine buildload
!
!
    ------------------------------------------------------------------------------

!
    subroutine buildstiff

        !! This subroutine builds the global stiffness matrix from
        !! the local element stiffness matrices

        use fedata
        use link1
        use plane42

        integer :: e, i, j
        integer :: nen
! Hint for system matrix in band form:
        integer :: irow, icol
        integer, dimension(mdim) :: edof
        real(wp), dimension(mdim) :: xe
        real(wp), dimension(mdim, mdim) :: ke
! Hint for modal analysis:
            real(wp), dimension(mdim, mdim) :: me
        real(wp) :: young, area
! Hint for modal analysis and continuum elements:
```

```
297         real(wp) :: nu, dens, thk
298
299         ! Reset stiffness matrix
300         kmat = 0
301         !if (.not. banded) then
302         !    kmat = 0
303         !else
304         !    kmat = 0
305         !end if
306
307         do e = 1, ne
308
309             ! Find coordinates and degrees of freedom
310             nen = element(e)%numnode
311             do i = 1, nen
312                 xe(2*i-1) = x(element(e)%ix(i),1)
313                 xe(2*i  ) = x(element(e)%ix(i),2)
314                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
315                 edof(2*i)   = 2 * element(e)%ix(i)
316             end do
317
318             ! Gather material properties and find element stiffness
                    matrix
319             select case( element(e)%id )
320             case( 1 )
321                 young = mprop(element(e)%mat)%young
322                 area  = mprop(element(e)%mat)%area
323                 call link1_ke(xe, young, area, ke)
324             case( 2 )
325                 young = mprop(element(e)%mat)%young
326                 nu = mprop(element(e)%mat)%nu
327                 thk = mprop(element(e)%mat)%thk
328                 call plane42_ke(xe, young, nu, thk, ke)
329             end select
330
331             ! Assemble into global matrix
332             if (.not. banded) then
333                 do i = 1, 2*nen
334                     do j = 1, 2*nen
335                         kmat(edof(i), edof(j)) = kmat(edof(i), edof(j))
                                + ke(i, j)
336                     end do
337                 end do
338 ! Hint: Can you eliminate the loops above by using a different Fortran
    array syntax?
339             else
340                 do i = 1, 2*nen
341                     do j = 1, 2*nen
342                     if (edof(j) >= edof(i)) then
```

```
343                         kmat(edof(j)-edof(i)+1, edof(i)) = kmat(edof(j)
                               -edof(i)+1, edof(i)) + ke(i, j)
344                     end if
345                     end do
346                 end do
347             end if
348         end do
349     end subroutine buildstiff
350 !
351 !
    ----------------------------------------------------------------------------

352 !
353     subroutine enforce
354
355         !! This subroutine enforces the support boundary conditions
356
357         use fedata
358
359         integer :: i, idof, k, r
360         real(wp) :: penal
361
362         ! Correct for supports
363         if (.not. banded) then
364                 if (.not. penalty) then
365                     do i = 1, nb
366                         idof = int(2*(bound(i,1)-1) + bound(i,2))
367                         p(1:neqn) = p(1:neqn) - kmat(1:neqn, idof) *
                               bound(i, 3)
368                         p(idof) = bound(i, 3)
369                         kmat(1:neqn, idof) = 0
370                         kmat(idof, 1:neqn) = 0
371                         kmat(idof, idof) = 1
372                     end do
373             else
374                     penal = penalty_fac*maxval(kmat)
375                     do i = 1, nb
376                         idof = int(2*(bound(i,1)-1) + bound(i,2))
377                         kmat(idof, idof) = kmat(idof, idof) + penal
378                         p(idof) = penal * bound(i, 3)
379                     end do
380             end if
381         else
382             if (.not. penalty) then
383                 do i = 1, nb
384                     idof = int(2*(bound(i,1)-1) + bound(i,2))
385
386                     do k = (idof-min(idof-1,bw-1)), (idof-1)
387                         P(k) = P(k) - kmat(idof+1-k, k) * bound(i, 3)
```

```
                        kmat(idof+1-k, k) = 0
                    end do

                    do r = 2, min(bw, neqn - idof + 1)
                        P(idof+r-1) = P(idof+r-1) - kmat(r, idof) *
                            bound(i, 3)
                        kmat(r, idof) = 0
                    end do

                    p(idof) = bound(i, 3)
                    kmat(1, idof) = 1
                end do
            else
                penal = penalty_fac*maxval(kmat)
                do i = 1, nb
                    idof = int(2*(bound(i,1)-1) + bound(i,2))
                    kmat(1, idof) = kmat(1, idof) + penal
                    p(idof) = penal * bound(i, 3)
                end do
            end if
        end if
    end subroutine enforce
!
!
    ------------------------------------------------------------------------------

!
    subroutine recover

        !! This subroutine recovers the element stress, element strain,
        !! and nodal reaction forces

        use fedata
        use link1
        use plane42

        integer :: e, i, nen
        integer :: edof(mdim)
        real(wp), dimension(mdim) :: xe, de
        real(wp), dimension(mdim, mdim) :: ke
        real(wp) :: young, area
! Hint for continuum elements:
        real(wp):: nu, dens, thk, sigma_vm, sigma_1, sigma_2, psi
        real(wp), dimension(3) :: estrain, estress

        ! Reset force vector
        p = 0

        do e = 1, ne
```

```
434
435             ! Find coordinates etc...
436             nen = element(e)%numnode
437             do i = 1,nen
438                 xe(2*i-1) = x(element(e)%ix(i), 1)
439                 xe(2*i)   = x(element(e)%ix(i), 2)
440                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
441                 edof(2*i)   = 2 * element(e)%ix(i)
442                 de(2*i-1) = d(edof(2*i-1))
443                 de(2*i)   = d(edof(2*i))
444             end do
445
446             ! Find stress and strain
447             select case( element(e)%id )
448             case( 1 )
449                 young = mprop(element(e)%mat)%young
450                 area  = mprop(element(e)%mat)%area
451                 call link1_ke(xe, young, area, ke)
452                 p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke(1:2*nen
                        ,1:2*nen), de(1:2*nen))
453                 call link1_ss(xe, de, young, estress, estrain)
454                 stress(e, 1:3) = estress
455                 strain(e, 1:3) = estrain
456                 case( 2 )
457                     young = mprop(element(e)%mat)%young
458                     nu = mprop(element(e)%mat)%nu
459                     thk = mprop(element(e)%mat)%thk
460                     call plane42_ke(xe, young, nu, thk, ke)
461                     p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke
                            (1:2*nen,1:2*nen), de(1:2*nen))
462                     call plane42_ss(xe, de, young, nu, estress, estrain
                            )
463                     stress(e, 1:3) = estress
464                     strain(e, 1:3) = estrain
465
466                     sigma_vm = sqrt((estress(1)**2) + (estress(2)**2) -
                            (estress(1)*estress(2)) + 3*(estress(3)**2))
467                     sigma_1 = 0.5 * (estress(1) + estress(2))+sqrt(((
                            estress(1)-estress(2))/2)**2 + (estress(3)**2))
468                     sigma_2 = 0.5 * (estress(1) + estress(2))-sqrt(((
                            estress(1)-estress(2))/2)**2 + (estress(3)**2))
469                     psi = atan2(((-2*estress(3))), ((estress(1)-estress
                            (2))))/2.0_wp
470
471                     stress_eq(e, 1) = sigma_vm
472                     stress_eq(e, 2) = sigma_1
473                     stress_eq(e, 3) = sigma_2
474                     stress_eq(e, 4) = psi
475             end select
```

```
476         end do
477     end subroutine recover
478
479
480 end module fea
```

## 3.2 plane42.f90

**Listing 4:** Finite element program

```
1  module plane42
2
3      !! This module contains subroutines specific to the plane42 element
4      !!
5      !! The plane42 element has 4 nodes. Each node has 2 degrees-of-
           freedom,
6      !! namely, displacement along the \(x\)- and \(y\)-coordinate
           directions.
7      !!
8      !! The nodes are numbered counter-clockwise as indicated below. The
9      !! figure also shows how the element edges are labelled. For
           example,
10     !! edge 1 is the edge between element node 1 and 2.
11     !!
12     !!        N4     E3     N3
13     !!           o------o
14     !!           |      |
15     !!        E4 |      | E2
16     !!           |      |
17     !!           o------o
18     !!        N1     E1     N2
19     !!
20     !!
21     !! `N1` = element node 1, `N2` = element node 2, etc
22     !! `E1` = element face 1, `E2` = element face 2, etc
23
24     use types
25     implicit none
26     save
27
28     private
29     public :: plane42_ke, plane42_re, plane42_ss
30
31 contains
32
33 ! Element stiffness matrix
34
35     subroutine plane42_ke(xe, young, nu, thk, ke)
36
```

```
37          ! Declare variables
38
39          real(wp) :: n(2,8), bmat(3,8)          ! N and B from shape
40          real(wp) :: cb(3,8)                    ! C*B, constitutive matrix
                multiplied with strain matrix
41          real(wp) :: jac(2,2), detjac           ! Jacobian matrix and
               determinant
42          integer  :: i, j                       ! loop counters over the
               Gauss points
43
44          real(wp), intent(in) :: young, nu, thk
45          real(wp), dimension(:), intent(in) :: xe
46          real(wp), dimension(:,:), intent(out) :: ke
47
48          real(wp) :: cmat(3,3), fact            ! constitutive matrix C (3
               x3) and factor E/(1-nu^2)
49          integer, parameter :: n_g = 2          ! Number of Gauss points
               in each direction
50          real(wp) :: gp(n_g), gw(n_g)           ! Gauss points and weights
51
52          ! Gauss points and weights
53          gp = [ -1.0_wp/sqrt(3.0_wp), 1.0_wp/sqrt(3.0_wp) ]
               ! Two gauss points from table under 8.2 in notes
54          gw = [ 1.0_wp, 1.0_wp]
               ! Gauss weights from table under 8.2 in notes
55
56          ! Constitutive matrix C for plane stress, Eq.(5.5) from Notes
               day 5-9.
57          ! Poisson between -1 < v < 1
58          cmat = 0
59          fact = young/(1-nu**2)   ! Outside the matrix
60          cmat(1,1) = fact
61          cmat(1,2) = fact*nu
62          cmat(2,1) = fact*nu
63          cmat(2,2) = fact
64          cmat(3,3) = fact*(1-nu)/2
65
66          ! Initialise ke to zero
67          ke = 0.0_wp
68
69
70          ! Double loop to calculate element stiffness matrix
71          do i = 1, n_g
72              do j = 1, n_g
73                  ! Call shape function to retrieve N, B, J and detJ at (
                       gp(i), gp(j)) = (xi, eta)
74                  call shape(xe, gp(i), gp(j), n, bmat, jac, detjac)
75                  ! Splitting up matmul because of potential stack
                       overflow problem
```

```
                cb = matmul(cmat, bmat)
                ! Calculate the element stiffness matrix
                ke = ke + gw(i)*gw(j)*thk*matmul(transpose(bmat), cb)*
                   detjac
            end do
        end do
    end subroutine
!
!
    ------------------------------------------------------------------------------

!
    subroutine plane42_re(xe, eface, fe, thk, re)

        !! This subroutine computes the element load vector due
        !! to surface traction (traction is always perpendicular
        !! to element face).

        integer, intent(in) :: eface
            !! Element face where traction (pressure) is applied

        real(wp), intent(in) :: fe
            !! Value of surface traction (pressure)
        real(wp), intent(in) :: thk
            !! Thickness of this element
        real(wp), dimension(:), intent(in) :: xe
            !! Nodal coordinates of this element in undeformed
               configuration (see also [[plane42_ke]])
        real(wp), intent(out) :: re(8)
            !! Element force vector
            !!
            !! * `re(1:2)` = \((f_x^1, f_y^1)\) force at element node 1
                in \(x\)- and y-direction
            !! * `re(3:4)` = \((f_x^2, f_y^2)\) force at element node 1
                in \(x\)- and y-direction
            !! * etc...
        real(wp) :: aa, bb, nface(2,8), f(2)

        aa = (xe(3)-xe(1))/2
        bb = (xe(8)-xe(2))/2

        nface = 0
        f = 0
        if (eface == 1) then
            f = [0.0_wp, fe]
            nface(1,1) = aa
            nface(1,3) = aa
            nface(2,2) = aa
            nface(2,4) = aa
```

```
119         elseif (eface == 2) then
120             f = [-fe, 0.0_wp]
121             nface(1,3) = bb
122             nface(1,5) = bb
123             nface(2,4) = bb
124             nface(2,6) = bb
125         elseif (eface == 3) then
126             f = [0.0_wp, -fe]
127             nface(1,5) = aa
128             nface(1,7) = aa
129             nface(2,6) = aa
130             nface(2,8) = aa
131         elseif (eface == 4) then
132             f = [fe, 0.0_wp]
133             nface(1,1) = bb
134             nface(1,7) = bb
135             nface(2,2) = bb
136             nface(2,8) = bb
137         endif
138         re = matmul(transpose(nface), f) * thk
139     end subroutine plane42_re
140 !
141 !
    ----------------------------------------------------------------------------

142 ! Stresses and strains
143     subroutine plane42_ss(xe, de, young, nu, estress, estrain)
144
145         !! This subrotuine computes the element stress and strain (The
                location inside the element
146         !! where stress and and strain is evaluated, is defined inside
                the subroutine).
147
148         real(wp), intent(in) :: young
149             !! Young's Modulus for this element
150         real(wp), intent(in) :: nu
151             !! Poisson's Ratio for this element
152         real(wp), dimension(:), intent(in)  :: xe
153             !! Nodal coordinates of this element in undeformed
                    configuration (see also [[plane42_ke]])
154         real(wp), dimension(:), intent(in)  :: de   ! de is element
                displacement and consists of 8 numbers that all belong to
                one element
155             !! Displacement field
156             !!
157             !! * `de(1:2)` = displacement of element node 1 along \(x\)
                    - and \(y\)-axis, respectively
158             !! * `de(3:4)` = displacement of element node 2 along \(x\)
                    - and \(y\)-axis, respectively
```

```
159             !! * etc...
160         real(wp), dimension(:), intent(out) :: estress
161             !! Stress at a point inside the element
162             !!
163             !! * `estress(1)` = \(\sigma_{11}\)
164             !! * `estress(2)` = \(\sigma_{22}\)
165             !! * `estress(3)` = \(\sigma_{12}\)
166         real(wp), dimension(:), intent(out) :: estrain
167             !! Strain at a point inside the element
168             !!
169             !! * `estrain(1)` = \(\epsilon_{11}\)
170             !! * `estrain(2)` = \(\epsilon_{22}\)
171             !! * `estrain(3)` = \(\epsilon_{12}\)
172
173         ! New variables
174         real(wp) :: bmat(3, 8), cmat(3, 3), n(2,8), jac(2,2), detjac,
                fact
175
176         ! Strain-displacement matrix B from the isoparametric
                formulation,
177         ! evaluated at one Gauss point (element centre, xi = eta = 0)
178         call shape(xe, 0.0_wp, 0.0_wp, n, bmat, jac, detjac)
179         ! Compute element strain
180         estrain = matmul(bmat, de)  ! (3x8)*8 = 3 numbers, so no danger
                of potential stack overflow
181
182         ! Build constitutive matrix (plane stress)
183         cmat = 0
184         fact = young/(1.0_wp-nu**2)
185         cmat(1,1) = fact
186         cmat(1,2) = fact*nu
187         cmat(2,1) = fact*nu
188         cmat(2,2) = fact
189         cmat(3,3) = fact*(1.0_wp-nu)/2
190
191         ! Compute element stress
192         estress = matmul(cmat, estrain)
193
194         ! Compute principal stress and direction
195         ! Does this in fea.f90 to not change the subroutine
196
197     end subroutine plane42_ss
198
199 !
        ----------------------------------------------------------------------------
200
201     subroutine shape(xe,xi,eta,n,bmat,jac,detjac)
202
```

```
203
204         real(wp), intent(in)  :: xe(8), xi, eta
205         real(wp), intent(out) :: n(2,8), bmat(3,8), jac(2,2), detjac
                ! Jacobimatrix is of size 2x2
206         real(wp) :: nv(4), dnxi(4), dneta(4), dnx(4), dny(4)
207         integer  :: i
208
209         nv    = 0.25_wp*(/ (1-xi)*(1-eta), (1+xi)*(1-eta), (1+xi)*(1+
               eta), (1-xi)*(1+eta) /) ! The Shape functions N1--N4
210         dnxi  = 0.25_wp*(/ -(1-eta), (1-eta), (1+eta), -(1+eta) /)
                                        ! Derivative with respect to xi
211         dneta = 0.25_wp*(/ -(1-xi), -(1+xi), (1+xi),  (1-xi)  /)
                                          ! Derivative with respect to eta
212
213         ! Jac(1,:) is row 1 in the matrix, and is for the derivative
               with respect to xi
214         jac(1,:) = (/ sum(dnxi *xe(1:7:2)), sum(dnxi *xe(2:8:2)) /)
                                  ! From index 1 till 7, with 2 timesteps
               , giving the x coordinates
215
216         ! Jac(2,:) is row 2 in the matrix, and is for the derivative
               with respect to eta
217         jac(2,:) = (/ sum(dneta*xe(1:7:2)), sum(dneta*xe(2:8:2)) /)
                                  ! From index 2 till 8, with 2 timesteps
               , giving the y coordinates
218         detjac = jac(1,1)*jac(2,2) - jac(1,2)*jac(2,1)       ! Solves
               the determinant for the 2x2 jacobi matrix
219
220         ! dN/dx og dN/dy via [Gamma] = [J]^-1 (inverted Jacobi matrix),
                from (8.5) in Notes day 5-9
221         dnx = ( jac(2,2)*dnxi - jac(1,2)*dneta)/detjac
222         dny = (-jac(2,1)*dnxi + jac(1,1)*dneta)/detjac
223
224
225         ! N matrix (2x8) is the shape function matrix (N1--N4 in a
               matrix)
226         ! {epsilon} = [L][gamma][N]{d} --> {epsilon} = [B_mat]{d} "
               Strain displacement matrix"
227         ! Loop goes through the four Nodes and places each nodes shape
               function in N and its derivatives in B
228         n = 0.0_wp; bmat = 0.0_wp
229         do i = 1, 4
230             n(1,2*i-1) = nv(i);   n(2,2*i) = nv(i)
231             bmat(1,2*i-1) = dnx(i)
232             bmat(2,2*i)   = dny(i)
233             bmat(3,2*i-1) = dny(i);  bmat(3,2*i) = dnx(i)
234         end do
235     end subroutine shape
236
```

```
237 end module plane42
```

# 4 Exercise 4 - Eigenvalue analysis

## 4.1 fea.f90

**Listing 5:** Finite element program

```
module fea

    !! This module contains procedures that are common to FEM-analysis

    implicit none
    save
    private
    public :: displ, initial, buildload, buildstiff, enforce, recover,
       compliance, eigen, mmul

contains

!
!
    ------------------------------------------------------------------------

!
    subroutine initial

        !! This subroutine is mainly used to allocate vectors and
           matrices

        use fedata
        use link1
        use plane42rectskewd

! Hint for continuum elements:
        !integer, parameter :: mdim = 8
        integer, dimension(mdim) :: edof
        integer :: e, i, nen

        ! This subroutine computes the number of global equation,
        ! half bandwidth, etc and allocates global arrays.

        ! Calculate number of equations
        neqn = 2*nn

        if (.not. banded) then
            allocate (kmat(neqn, neqn))
        else
            ! Making sure bw = 0
            bw = 0
            ! Looping through the different elements
            do e = 1, ne
```

```
41              nen = element(e)%numnode
42                  ! Picking out the different edofs
43                  do i = 1, nen
44                      edof(2*i-1) = 2 * element(e)%ix(i) - 1
45                      edof(2*i)   = 2 * element(e)%ix(i)
46                  end do
47                  ! Updating bw if the bandwith of the element is bigger
                        than the previous ones
48                  bw = max(bw, (maxval(edof(1:2*nen)) - minval(edof(1:2*
                        nen)) + 1))
49              end do
50              allocate (kmat(bw, neqn))
51          end if
52          allocate (p(neqn), d(neqn))
53          allocate (strain(ne, 3), stress(ne, 3))
54          allocate (stress_eq(ne, 4))
55
56          ! Initial stress and strain
57          strain = 0
58          stress = 0
59          stress_eq = 0
60      end subroutine initial
61  !
62  !
        ------------------------------------------------------------------------------

63  !
64      subroutine displ
65
66          !! This subroutine calculates displacements
67
68          use fedata
69          use numeth
70          use processor
71
72          integer :: e
73          real(wp), dimension(:), allocatable :: plotval
74
75          ! Build load-vector
76          call buildload
77
78          ! Build stiffness matrix
79          call buildstiff
80
81          ! Remove rigid body modes
82          call enforce
83
84          if (.not. banded) then
85              ! Factor stiffness matrix
```

```
86              call factor(kmat)
87              ! Solve for displacement vector
88              call solve(kmat, p)
89          else
90              ! Factor banded stiffness matrix
91              call bfactor(kmat)
92              ! Solve for displacement vector
93              call bsolve(kmat, p)
94          end if
95
96          ! Transfer results
97          d(1:neqn) = p(1:neqn)
98
99          ! Recover stress
100         call recover
101
102         ! Output results
103         call output
104
105         ! Plot deformed structure
106         call plotmatlabdef('Deformed')
107
108         ! Plot element values
109         allocate (plotval(ne))
110         do e = 1, ne
111             ! Allocated different stresses to elements
112             if (element(e)%id == 1) then
113                 plotval(e) = stress(e,1)
114             ! Allocated different stresses to elements
115             else if (element(e)%id == 2) then
116                 plotval(e) = stress(e,3)
117             end if
118         end do
119         call plotmatlabeval('Shear␣stress',plotval)
120         call plotmatlabeval('von␣Mises', stress_eq(:, 1))
121         call plotmatlabevec('Principal␣stresses', stress_eq(:, 2),
               stress_eq(:, 3), stress_eq(:, 4))
122
123     end subroutine displ
124 !
125 !
    ----------------------------------------------------------------------------

126 !
127     subroutine eigen
128
129         !! This subroutine calculates displacements
130
131         use fedata
```

```
132         use numeth
133         use processor
134 
135         integer :: e, i, j, neig, n_c, idof, niter_max, niter
136         real(wp) :: c, tol, rp
137         real(wp), dimension(:), allocatable :: plotval
138         real(wp), dimension(:), allocatable :: xv, yv, lambda, xv_old,
               yv_old
139         real(wp), dimension(:,:), allocatable :: zmat
140         logical, allocatable :: is_fixed(:)
141         ! For the plotting
142         real(wp) :: len_model, scale_mode, tvec(2)
143 
144         ! Give vector right dimensions
145         allocate(xv(neqn), yv(neqn), is_fixed(neqn), xv_old(neqn),
               yv_old(neqn))
146 
147         ! Scheme to count amount of unique boundary conditions
148         is_fixed = .false.
149         ! Limit of p-iterations
150         niter_max = 300
151         ! Tolerance criterion
152         tol = 1.0e-10_wp
153         ! Iterates through number of boundary condtitions
154         do i = 1, nb
155             ! Translates the boundary condition to global DOF
156             idof = int(2*(bound(i,1)-1) + bound(i,2))
157             ! Sets DOF true to count up amount of unique BC's
158             is_fixed(idof) = .true.
159         end do
160         ! Counts up amount of unique BC's
161         n_c = count(is_fixed)
162         ! Calculates number of eigenfrequencies
163         neig = neqn - n_c
164         ! With many modes, the later modes are computationally heavy
               and innaccurate
165         neig = min(neig, 10)
166 
167         ! After neig calculation, give the rest the right values
168         allocate(dmat(neqn, neig), zmat(neqn, neig), lambda(neig),
               omega(neig))
169         ! Allocating me_all: the element mass matrices
170         allocate(me_all(mdim, mdim, ne))
171         ! Resetting value
172         me_all = 0.0_wp
173 
174         ! Build load-vector
175         call buildload
176 
```

```
177         ! Build stiffness matrix
178         call buildstiff
179
180         ! Remove rigid body modes
181         call enforce
182
183         if (.not. banded) then
184             ! Factor stiffness matrix
185             call factor(kmat)
186         else
187             ! Factor banded stiffness matrix
188             call bfactor(kmat)
189         end if
190
191         do i = 1, neig
192             xv = 1.0_wp
193             call mmul(xv, yv)
194             do j = 1, i-1
195                 call mmul(dmat(:,j), zmat(:,j))
196             end do
197             ! Iterate
198             do niter = 1, niter_max
199                 where (is_fixed) yv = 0.0_wp
200                 xv_old = xv
201                 xv = yv
202                 call bsolve(kmat, xv)
203                 do j = 1, i-1
204                     ! Calculate c
205                     c = dot_product(xv, zmat(:,j))
206                     ! Orthogonalize c
207                     xv = xv - c * dmat(:, j)
208                 end do
209                 ! Save Y_{p-1}-value for later
210                 yv_old = yv
211                 ! Calculate new Y-value
212                 call mmul(xv, yv)
213                 ! Calculate rp
214                 rp = sqrt(dot_product(xv, yv))
215                 ! Calculate Y
216                 yv = yv/rp
217                 ! Check with converge criterion
218                 if ((sqrt(dot_product(xv - xv_old, xv - xv_old)) / sqrt
                        (dot_product(xv, xv))) < tol) then
219                     exit
220                 end if
221             end do
222             ! Give warning if loop did not converge
223             if (niter > niter_max) then
```

```
224                 print *, 'WARNING: mode', i, 'did not converge within',
                        niter_max, 'iterations.'
225             end if
226             ! Calculate eigenvector D
227             dmat(:, i) = xv/rp
228             ! Calculate Eigenvalue
229             lambda(i) = dot_product(xv, yv_old)/(rp**2)
230             omega(i) = sqrt(lambda(i))
231             print *, i, lambda(i), omega(i), omega(i)/(2.0_wp*acos(-1.0
                    _wp))
232         end do
233
234         ! Plot the eigenmodes
235         ! Calculate length - only works for specific problems
236         len_model = maxval(x(:,1)) - minval(x(:,1))
237         ! Plot certain eigenmodes: here we choose 3
238         do i = 1, 3
239             ! Scale the model, so its easy to observe
240             ! Scale it to 0.1 * length of model each time
241             ! Dividing by the absolute maximum value of each modes
                    eigenvector
242             scale_mode = 0.1_wp * len_model/(maxval(abs(dmat(:, i))))
243             ! Time vectors:
244             ! Total time for animation
245             tvec(1) = 2.0_wp*(2.0_wp*pi/omega(i))
246             ! Time between each time instance
247             tvec(2) = (2.0_wp*pi/omega(i))/30.0_wp
248             call plotmatlabeig('Mode', omega(i), scale_mode*dmat(:, i),
                    tvec)
249         end do
250     end subroutine eigen
251 !
252 !
    ----------------------------------------------------------------------------

253 !
254     subroutine compliance
255
256         !! This subroutine calculates compliance
257
258         use fedata
259
260         real(wp) :: c
261
262         c = dot_product(p, d)
263         write (*, '("Compliance C = p^T d = ", es15.8)') c
264         open (20, file='compliance.dat', position='append')
265         write (20, '(i8, 1x, i8, 1x, es20.10)') ne, neqn, c
266         close (20)
```

```
267
268     end subroutine compliance
269
270
271 !
272 !   ------------------------------------------------------------------------------
273 !
274     subroutine buildload
275
276         !! This subroutine builds the global load vector
277
278         use fedata
279         use plane42rectskewd
280
281 ! Hint for continuum elements:
282
283         integer :: i, j, e, nen, idof, eface
284         real(wp) :: fe, thk
285         integer, dimension(mdim) :: edof
286         real(wp), dimension(mdim) :: xe
287         real(wp), dimension(mdim) :: re
288
289         ! Build load vector
290         p(1:neqn) = 0
291         do i = 1, np
292             select case(int(loads(i, 1)))
293             case( 1 )
294                 ! Build nodal load contribution
295                 idof = int(2*(loads(i,2)-1) + loads(i,3))
296                 p(idof) = p(idof) + loads(i, 4)
297             case( 2 )
298                 ! Build uniformly distributed surface (pressure) load
                      contribution
299                 e = int(loads(i, 2))
300                 eface = int(loads(i,3))
301                 fe    = loads(i,4)
302                 thk   = mprop(element(e)%mat)%thk
303                 nen   = element(e)%numnode
304
305                 do j = 1, nen
306                     xe(2*j-1) = x(element(e)%ix(j),1)
307                     xe(2*j  ) = x(element(e)%ix(j),2)
308                     edof(2*j-1) = 2 * element(e)%ix(j) - 1
309                     edof(2*j)   = 2 * element(e)%ix(j)
310                 end do
311                 call plane42rect_re(xe, eface, fe, thk, re)
312                 p(edof(1: 2*nen)) = p(edof(1: 2*nen)) + re(1:2*nen)
```

```
313             case default
314                 print *, 'ERROR in fea/buildload 2'
315                 print *, 'Load type not known'
316                 stop
317             end select
318         end do
319     end subroutine buildload
320 !
321 !
    ------------------------------------------------------------------------------

322 !
323     subroutine buildstiff
324 
325         !! This subroutine builds the global stiffness matrix from
326         !! the local element stiffness matrices
327 
328         use fedata
329         use link1
330         use plane42rectskewd
331 
332         integer :: e, i, j
333         integer :: nen
334 ! Hint for system matrix in band form:
335         integer :: irow, icol
336         integer, dimension(mdim) :: edof
337         real(wp), dimension(mdim) :: xe
338         real(wp), dimension(mdim, mdim) :: ke
339 ! Hint for modal analysis:
340         real(wp), dimension(mdim, mdim) :: me
341         real(wp) :: young, area
342 ! Hint for modal analysis and continuum elements:
343         real(wp) :: nu, dens, thk
344 
345         ! Reset stiffness matrix
346         kmat = 0
347         !if (.not. banded) then
348         !    kmat = 0
349         !else
350         !    kmat = 0
351         !end if
352 
353         do e = 1, ne
354 
355             ! Find coordinates and degrees of freedom
356             nen = element(e)%numnode
357             do i = 1, nen
358                 xe(2*i-1) = x(element(e)%ix(i),1)
359                 xe(2*i  ) = x(element(e)%ix(i),2)
```

```
360                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
361                 edof(2*i)   = 2 * element(e)%ix(i)
362             end do
363
364             ! Gather material properties and find element stiffness
                    matrix
365             select case( element(e)%id )
366             case( 1 )
367                 young = mprop(element(e)%mat)%young
368                 area  = mprop(element(e)%mat)%area
369                 call link1_ke(xe, young, area, ke)
370             case( 2 )
371                 young = mprop(element(e)%mat)%young
372                 nu = mprop(element(e)%mat)%nu
373                 thk = mprop(element(e)%mat)%thk
374                 dens = mprop(element(e)%mat)%dens
375                 call plane42rect_ke(xe, young, nu, thk, ke, dens, me)
376                 ! Save element mass matrix to not calculate sitffnesses
                        and masses
377                 ! too many times
378                 me_all(:,:,e) = me
379             end select
380
381             ! Assemble into global matrix
382             if (.not. banded) then
383                 do i = 1, 2*nen
384                     do j = 1, 2*nen
385                         kmat(edof(i), edof(j)) = kmat(edof(i), edof(j))
                                + ke(i, j)
386                     end do
387                 end do
388 ! Hint: Can you eliminate the loops above by using a different Fortran
       array syntax?
389             else
390                 do i = 1, 2*nen
391                     do j = 1, 2*nen
392                     if (edof(j) >= edof(i)) then
393                         kmat(edof(j)-edof(i)+1, edof(i)) = kmat(edof(j)
                               -edof(i)+1, edof(i)) + ke(i, j)
394                     end if
395                     end do
396                 end do
397             end if
398         end do
399     end subroutine buildstiff
400 !
401 !
    ------------------------------------------------------------------------
```

```
!
    subroutine mmul(invector, outvector)

        !! This subroutine multiplicates the mass matrix and
           eigenvectors

        use fedata

! Declaring input and output vectors
        real(wp), dimension(:), intent(in) :: invector
        real(wp), dimension(:), intent(out) :: outvector
! Declaring necessary integers for loops
        integer :: e, i, j, nen
        integer, dimension(mdim) :: edof
        real(wp), dimension(mdim, mdim) :: me

        ! Resetting the outvector
        outvector = 0.0_wp
        do e = 1, ne
            ! Find number of nodes in a given element
            nen = element(e)%numnode
            ! Iterate through nodes in element
            do i = 1, nen
                ! Element degree of freedom,
                edof(2*i-1) = 2 * element(e)%ix(i) - 1
                edof(2*i)   = 2 * element(e)%ix(i)
            end do
            ! Adding the different element products together
            outvector(edof(1:2*nen)) = outvector(edof(1:2*nen)) +
               matmul(me_all(1:2*nen, 1:2*nen, e), invector(edof(1:2*
               nen)))
        end do

    end subroutine mmul
!
!
    ------------------------------------------------------------------------------

!
    subroutine enforce

        !! This subroutine enforces the support boundary conditions

        use fedata

        integer :: i, idof, k, r
        real(wp) :: penal

        ! Correct for supports
```

```
446         if (.not. banded) then
447             if (.not. penalty) then
448                 do i = 1, nb
449                     idof = int(2*(bound(i,1)-1) + bound(i,2))
450                     p(1:neqn) = p(1:neqn) - kmat(1:neqn, idof) *
                            bound(i, 3)
451                     p(idof) = bound(i, 3)
452                     kmat(1:neqn, idof) = 0
453                     kmat(idof, 1:neqn) = 0
454                     kmat(idof, idof) = 1
455                 end do
456             else
457                 penal = penalty_fac*maxval(kmat)
458                 do i = 1, nb
459                     idof = int(2*(bound(i,1)-1) + bound(i,2))
460                     kmat(idof, idof) = kmat(idof, idof) + penal
461                     p(idof) = penal * bound(i, 3)
462                 end do
463             end if
464         else
465             if (.not. penalty) then
466                 do i = 1, nb
467                     idof = int(2*(bound(i,1)-1) + bound(i,2))
468
469                     do k = (idof-min(idof-1, bw-1)), (idof-1)
470                         P(k) = P(k) - kmat(idof+1-k, k) * bound(i, 3)
471                         kmat(idof+1-k, k) = 0
472                     end do
473
474                     do r = 2, min(bw, neqn - idof + 1)
475                         P(idof+r-1) = P(idof+r-1) - kmat(r, idof) *
                                bound(i, 3)
476                         kmat(r, idof) = 0
477                     end do
478
479                     p(idof) = bound(i, 3)
480                     kmat(1, idof) = 1
481                 end do
482             else
483                 penal = penalty_fac*maxval(kmat)
484                 do i = 1, nb
485                     idof = int(2*(bound(i,1)-1) + bound(i,2))
486                     kmat(1, idof) = kmat(1, idof) + penal
487                     p(idof) = penal * bound(i, 3)
488                 end do
489             end if
490         end if
491     end subroutine enforce
492 !
```

```
493 !
    ------------------------------------------------------------------------------
494 !
495     subroutine recover
496
497         !! This subroutine recovers the element stress, element strain,
498         !! and nodal reaction forces
499
500         use fedata
501         use link1
502         use plane42rectskewd
503
504         integer :: e, i, nen
505         integer :: edof(mdim)
506         real(wp), dimension(mdim) :: xe, de
507         real(wp), dimension(mdim, mdim) :: ke
508         real(wp) :: young, area
509 ! Hint for continuum elements:
510         real(wp):: nu, dens, thk, sigma_vm, sigma_1, sigma_2, psi
511         real(wp), dimension(3) :: estrain, estress
512         real(wp), dimension(mdim, mdim) :: me
513
514         ! Reset force vector
515         p = 0
516
517         do e = 1, ne
518
519             ! Find coordinates etc...
520             nen = element(e)%numnode
521             do i = 1,nen
522                 xe(2*i-1) = x(element(e)%ix(i), 1)
523                 xe(2*i)   = x(element(e)%ix(i), 2)
524                 edof(2*i-1) = 2 * element(e)%ix(i) - 1
525                 edof(2*i)   = 2 * element(e)%ix(i)
526                 de(2*i-1) = d(edof(2*i-1))
527                 de(2*i)   = d(edof(2*i))
528             end do
529
530             ! Find stress and strain
531             select case( element(e)%id )
532             case( 1 )
533                 young = mprop(element(e)%mat)%young
534                 area  = mprop(element(e)%mat)%area
535                 call link1_ke(xe, young, area, ke)
536                 p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke(1:2*nen
                        ,1:2*nen), de(1:2*nen))
537                 call link1_ss(xe, de, young, estress, estrain)
538                 stress(e, 1:3) = estress
```

```
                strain(e, 1:3) = estrain
                case( 2 )
                    young = mprop(element(e)%mat)%young
                    nu = mprop(element(e)%mat)%nu
                    thk = mprop(element(e)%mat)%thk
                    dens = mprop(element(e)%mat)%dens
                    call plane42rect_ke(xe, young, nu, thk, ke, dens,
                       me)
                    p(edof(1:2*nen)) = p(edof(1:2*nen)) + matmul(ke
                       (1:2*nen,1:2*nen), de(1:2*nen))
                    call plane42rect_ss(xe, de, young, nu, estress,
                       estrain)
                    stress(e, 1:3) = estress
                    strain(e, 1:3) = estrain

                    ! Von mises
                    sigma_vm = sqrt((estress(1)**2) + (estress(2)**2) -
                        (estress(1)*estress(2)) + 3*(estress(3)**2))
                    ! Sigma 1
                    sigma_1 = 0.5 * (estress(1) + estress(2))+sqrt(((
                       estress(1)-estress(2))/2)**2 + (estress(3)**2))
                    ! Sigma 2
                    sigma_2 = 0.5 * (estress(1) + estress(2))-sqrt(((
                       estress(1)-estress(2))/2)**2 + (estress(3)**2))
                    ! Angle
                    psi = atan2(((-2*estress(3))), ((estress(1)-estress
                       (2))))/2.0_wp

                    stress_eq(e, 1) = sigma_vm
                    stress_eq(e, 2) = sigma_1
                    stress_eq(e, 3) = sigma_2
                    stress_eq(e, 4) = psi
            end select
        end do
    end subroutine recover

end module fea
```

## 4.2 plane42rectskewd.f90

**Listing 6:** Finite element program

```
module plane42rectskewd

    !! This module contains subroutines specific to the plane42 element
    !!
    !! The plane42 element has 4 nodes. Each node has 2 degrees-of-
       freedom,
```

```
6       !! namely, displacement along the \(x\)- and \(y\)-coordinate
            directions.
7       !!
8       !! The nodes are numbered counter-clockwise as indicated below. The
9       !! figure also shows how the element edges are labelled. For
            example,
10      !! edge 1 is the edge between element node 1 and 2.
11      !!
12      !!        N4     E3     N3
13      !!           o------o
14      !!           |      |
15      !!        E4 |      | E2
16      !!           |      |
17      !!           o------o
18      !!        N1     E1     N2
19      !!
20      !!
21      !! `N1` = element node 1, `N2` = element node 2, etc
22      !! `E1` = element face 1, `E2` = element face 2, etc
23
24      use types
25      implicit none
26      save
27
28      private
29      public :: plane42rect_ke, plane42rect_re, plane42rect_ss
30
31  contains
32
33      subroutine plane42rect_shape(ex, ey, xi, eta, n, B, J_det, J)
34          implicit none
35
36          real(wp), intent(in) :: ex(4), ey(4), xi, eta
37          real(wp), intent(out) :: B(3, 8), J_det, J(2, 2), n(2,8)
38
39          real(wp) :: N_thilde(4, 8), L(3, 4), Gamma(2, 2), Gamma_thilde
                (4, 4)
40          real(wp) :: xi_i(4) = [-1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp]
41          real(wp) :: eta_i(4) = [-1.0_wp, -1.0_wp, 1.0_wp, 1.0_wp]
42          real(wp) :: dNdxi(4), dNdeta(4)
43          integer :: i
44
45          ! Defined in (8.6) in courese notes
46          L = 0.0_wp
47          L(1, 1) = 1
48          L(2, 4) = 1
49          L(3,2) = 1
50          L(3,3) = 1
51
```

```
52          ! Resetting values
53          n = 0.0_wp
54          N_thilde = 0.0_wp
55
56          ! Building the N_thilde and n matrices given in DAY8
57          do i = 1, 4
58              dNdxi(i) = 0.25_wp * xi_i(i) * (1 + eta_i(i) * eta)
59              dNdeta(i) = 0.25_wp * eta_i(i) * (1 + xi_i(i) * xi)
60              N_thilde(1, 2*i-1) = dNdxi(i)
61              N_thilde(2, 2*i-1) = dNdeta(i)
62              N_thilde(3, 2*i) = dNdxi(i)
63              N_thilde(4, 2*i) = dNdeta(i)
64              n(1, 2*i-1) = 0.25_wp * (1+xi_i(i)*xi)*(1+eta_i(i)*eta)
65              n(2, 2*i) = 0.25_wp * (1+xi_i(i)*xi)*(1+eta_i(i)*eta)
66          end do
67          ! Defining the Jacobi matrix
68          J(1, 1) = sum(dNdxi * ex)
69          J(2, 1) = sum(dNdeta * ex)
70          J(1, 2) = sum(dNdxi * ey)
71          J(2, 2) = sum(dNdeta * ey)
72
73          ! Defining the Jacobi determinant and the gamma (inverted
                Jacobi)
74          J_det = J(1, 1) * J(2, 2) - J(2, 1) * J(1, 2)
75          Gamma(1, 1) = J(2, 2) / J_det
76          Gamma(2, 1) = -J(2, 1) / J_det
77          Gamma(1, 2) = -J(1, 2) / J_det
78          Gamma(2, 2) = J(1, 1) / J_det
79
80          ! Defining Gamma_thilde
81          Gamma_thilde = 0.0_wp
82          Gamma_thilde(1:2, 1:2) = Gamma
83          Gamma_thilde(3:4, 3:4) = Gamma
84
85          B = matmul(L, matmul(Gamma_thilde, N_thilde))
86
87      end subroutine plane42rect_shape
88  !
89  !
        ----------------------------------------------------------------------------------------------------

90  !
91
92      subroutine plane42rect_ke(xe, young, nu, thk, ke, dens, me)
93
94          !! This subroutine constructs the stiffness matrix for
95          !! a rectangular 4-noded quad element.
96          real(wp), intent(in) :: young
97              !! Young's Modulus for this element
```

```
98          real(wp), intent(in) :: nu
99              !! Poisson's Ratio for this element
100         real(wp), intent(in) :: thk
101             !! Thickness of this element
102         real(wp), intent(in) :: dens
103             !! Density of this element
104         real(wp), dimension(:), intent(in) :: xe
105             !! Nodal coordinates of this element in undeformed
                configuration
106             !!
107             !! * `xe(1:2)` = \((x,y)\)-coordinates of element node 1
108             !! * `xe(3:4)` = \((x,y)\)-coordinates of element node 1
109             !! * `xe(5:6)` = \((x,y)\)-coordinates of element node 2
110             !! * `xe(7:8)` = \((x,y)\)-coordinates of element node 2
111             !!
112             !! See also [[plane42rect]]
113         real(wp), dimension(:,:), intent(out) :: ke
114         real(wp), dimension(:,:), intent(out) :: me
115         real(wp) :: cmat(3,3), fact
116         real(wp), parameter :: gauss_points(2) = [-1.0_wp/sqrt(3.0_wp),
                1.0_wp/sqrt(3.0_wp)]
117         real(wp), parameter :: gauss_weight(2) = [1.0_wp, 1.0_wp]
118         real(wp) :: ex(4), ey(4)
119         real(wp) :: B(3, 8), J_det, J(2, 2), n(2, 8), CB(3, 8)
120         integer :: i, y, n_g
121
122         ! Resetting element stiffness matrix
123         ke = 0.0_wp
124         me = 0.0_wp
125
126         ! build constitutive matrix (plane stress)
127         cmat = 0.0_wp
128         fact = young/(1-nu**2)
129         cmat(1,1) = fact
130         cmat(1,2) = fact*nu
131         cmat(2,1) = fact*nu
132         cmat(2,2) = fact
133         cmat(3,3) = fact*(1-nu)/2
134
135         ! Gaus-points, 2*n_g - 1 = p, p = 2 => n_g >= 1.5 => n_g = 2
136         n_g = 2
137         ! Picking out the x and y coordinates, they are every other
              term each
138         ex = xe(1::2)
139         ey = xe(2::2)
140
141         ! Double loop to calculate element stiffness matrix
142         do i = 1, n_g
143             ! y because fortran doesnt care about capitol letters
```

```
144             do y = 1, n_g
145                 ! Call shape function to retrieve B, J_det and J
146                 call plane42rect_shape(ex, ey, gauss_points(i),
                        gauss_points(y), n, B, J_det, J)
147                 ! Splitting up matmul because of potential stack
                        overflow problem
148                 CB = matmul(cmat, B)
149                 ! Calculate the element stiffness matrix
150                 ke = ke + gauss_weight(i)*gauss_weight(y)*thk*matmul(
                        transpose(B), CB)*J_det
151                 me = me + gauss_weight(i)*gauss_weight(y)*dens*thk*
                        matmul(transpose(n), n)*J_det
152             end do
153         end do
154
155     end subroutine plane42rect_ke
156 !
157 !
    ----------------------------------------------------------------------------

158 !
159     subroutine plane42rect_re(xe, eface, fe, thk, re)
160
161         !! This subroutine computes the element load vector due
162         !! to surface traction (traction is always perpendicular
163         !! to element face).
164         !! Consistent nodal loads
165
166         integer, intent(in) :: eface
167             !! Element face where traction (pressure) is applied
168
169         real(wp), intent(in) :: fe
170             !! Value of surface traction (pressure)
171         real(wp), intent(in) :: thk
172             !! Thickness of this element
173         real(wp), dimension(:), intent(in) :: xe
174             !! Nodal coordinates of this element in undeformed
                    configuration (see also [[plane42rect_ke]])
175         real(wp), intent(out) :: re(8)
176             !! Element force vector
177             !!
178             !! * `re(1:2)` = \((f_x^1, f_y^1)\) force at element node 1
                     in \(x\)- and y-direction
179             !! * `re(3:4)` = \((f_x^2, f_y^2)\) force at element node 1
                     in \(x\)- and y-direction
180             !! * etc...
181         real(wp), parameter :: gauss_weight(2) = [1.0_wp, 1.0_wp]
182         real(wp) :: ex(4), ey(4)
183         real(wp) :: B(3, 8), J_det, J(2, 2), J_cal(2), n(2, 8)
```

```
184         real(wp), parameter :: gauss_points(2) = [-1.0_wp/sqrt(3.0_wp),
                1.0_wp/sqrt(3.0_wp)]
185         integer :: i
186         integer, parameter :: n_g = 2
187 
188         ex = xe(1::2)
189         ey = xe(2::2)
190         J_cal = 0.0_wp
191         re = 0.0_wp
192 
193         ! Checking which face of the element we are at
194         if (eface == 1) then
195             ! iterating through Gauss points
196             do i = 1, n_g
197                 call plane42rect_shape(ex, ey, gauss_points(i), -1.0_wp
                        , n, B, J_det, J)
198                 ! Defining the J-vector for consistant nodal loads
199                 J_cal(1) = -J(1, 2)
200                 J_cal(2) = J(1, 1)
201                 re = re + gauss_weight(i) * thk * fe * matmul(transpose
                        (n), J_cal)
202             end do
203         elseif (eface == 2) then
204             ! iterating through Gauss points
205             do i = 1, n_g
206                 call plane42rect_shape(ex, ey, 1.0_wp, gauss_points(i),
                         n, B, J_det, J)
207                 ! Defining the J-vector for consistant nodal loads
208                 J_cal(1) = -J(2, 2)
209                 J_cal(2) = J(2, 1)
210                 re = re + gauss_weight(i) * thk * fe * matmul(transpose
                        (n), J_cal)
211             end do
212         elseif (eface == 3) then
213             ! iterating through Gauss points
214             do i = 1, n_g
215                 call plane42rect_shape(ex, ey, gauss_points(i), 1.0_wp,
                         n, B, J_det, J)
216                 ! Defining the J-vector for consistant nodal loads
217                 J_cal(1) = J(1, 2)
218                 J_cal(2) = -J(1, 1)
219                 re = re + gauss_weight(i) * thk * fe * matmul(transpose
                        (n), J_cal)
220             end do
221         elseif (eface == 4) then
222             ! iterating through Gauss points
223             do i = 1, n_g
224                 call plane42rect_shape(ex, ey, -1.0_wp, gauss_points(i)
                        , n, B, J_det, J)
```

```
225                 ! Defining the J-vector for consistant nodal loads
226                 J_cal(1) = J(2, 2)
227                 J_cal(2) = -J(2, 1)
228                 re = re + gauss_weight(i) * thk * fe * matmul(transpose
                        (n), J_cal)
229             end do
230         endif
231     end subroutine plane42rect_re
232 !
233 !
    ----------------------------------------------------------------------------

234 !
235     subroutine plane42rect_ss(xe, de, young, nu, estress, estrain)
236
237         !! This subrotuine computes the element stress and strain (The
                location inside the element
238         !! where stress and and strain is evaluated, is defined inside
                the subroutine).
239
240         real(wp), intent(in) :: young
241             !! Young's Modulus for this element
242         real(wp), intent(in) :: nu
243             !! Poisson's Ratio for this element
244         real(wp), dimension(:), intent(in)  :: xe
245             !! Nodal coordinates of this element in undeformed
                    configuration (see also [[plane42rect_ke]])
246         real(wp), dimension(:), intent(in)  :: de
247             !! Displacement field
248             !!
249             !! * `de(1:2)` = displacement of element node 1 along \(x\)
                    - and \(y\)-axis, respectively
250             !! * `de(3:4)` = displacement of element node 2 along \(x\)
                    - and \(y\)-axis, respectively
251             !! * etc...
252         real(wp), dimension(:), intent(out) :: estress
253             !! Stress at a point inside the element
254             !!
255             !! * `estress(1)` = \(\sigma_{11}\)
256             !! * `estress(2)` = \(\sigma_{22}\)
257             !! * `estress(3)` = \(\sigma_{12}\)
258         real(wp), dimension(:), intent(out) :: estrain
259             !! Strain at a point inside the element
260             !!
261             !! * `estrain(1)` = \(\epsilon_{11}\)
262             !! * `estrain(2)` = \(\epsilon_{22}\)
263             !! * `estrain(3)` = \(\epsilon_{12}\)
264
265         ! New variables
```

```
266         real(wp) :: B(3, 8), cmat(3, 3), ex(4), ey(4), n(2, 8), J(2, 2)
267         real(wp) :: fact, J_det, xi, eta
268 
269         ex = xe(1::2)
270         ey = xe(2::2)
271         xi = 0.0_wp
272         eta = 0.0_wp
273 
274         ! Build strain-displacement matrix
275 
276         call plane42rect_shape(ex, ey, xi, eta, n, B, J_det, J)
277 
278         ! Compute element strain
279         estrain = matmul(B, de)
280 
281         ! Build constitutive matrix (plane stress)
282         cmat = 0
283         fact = young/(1.0_wp-nu**2)
284         cmat(1,1) = fact
285         cmat(1,2) = fact*nu
286         cmat(2,1) = fact*nu
287         cmat(2,2) = fact
288         cmat(3,3) = fact*(1.0_wp-nu)/2
289 
290         ! Compute element stress
291         estress = matmul(cmat, estrain)
292 
293         ! Compute principal stress and direction
294         ! Does this in fea.f90 to not change the subroutine
295 
296     end subroutine plane42rect_ss
297 
298 end module plane42rectskewd
```